NTNU | Norwegian University of Science and Technology

Exercise 2

AI-assisted coding

TDT4242 Advanced Software Engineering

Group 3
Raffaella Corcione
Ruggero Facchini
Birgitte Louise Vik Thoresen

Spring 2026

Contents

1 Task 2.1. Prompting strategy for code generation **2**
1.1 Selected group of requirements . . . 2
1.2 Prompt Strategy . . . 3
1.3 Example Prompt . . . 3

2 Task 2.2. AI-Based Code Generation **7**
2.1 Installation and configuration . . . 7
2.1.1 Environment Configuration . . . 7
2.1.2 Installation Steps . . . 7
2.1.3 Running the Project . . . 8
2.2 Revision of AI-generated code . . . 8
2.3 Explanation of the codebase structure and design decisions . . . 8
2.3.1 Frontend components . . . 9
2.3.2 Backend components . . . 9
2.3.3 Example data flow: form submission process . . . 10
2.4 Screenshots of successful execution . . . 11

3 Task 2.3. Code Review **13**
3.1 Large Class . . . 13
3.2 Duplicated Code . . . 13
3.3 Primitive Obsession . . . 14
3.4 Dead/unreachable code . . . 15
3.5 Duplicated logic . . . 16
3.6 Other . . . 18

Chapter 1

Task 2.1. Prompting strategy for code generation

For this task, one self-contained group of requirements was selected. From the requirements and dependency map from Task 1.3, the group selected was "Assignment declaration".

1.1 Selected group of requirements

The requirements related to the selected group are listed below. Here are the functional requirements.

- As a student, I want to declare AI usage in assignments by filling out a form, so that I can ensure compliance with the ethical guidelines provided by the institution. (FR3)
- As a student, I want to specify specific tools used in each assignment and their purpose, so that I can achieve transparency in my work. (FR4)
- As a student, I want to be able to select the tools I used during assignment declaration from a list (ChatGPT, GitHub Copilot, Claude, Grammarly, Turnitin, DALL-E, data analysis tools), so that I can perform the declaration quickly. (FR5)
- As a student, I want to have the option to declare AI tools that are not present in the list during assignment declaration, so that I can feel free to use the tools that are most useful to me while still being transparent. (FR6)
- As a student, I want to declare the content generated by AI in my assignments, so that I can be transparent in my work. (FR7)
- As a student, I want to attach screenshots or examples of AI-generated content to my declaration, so that I can provide clear documentation. (FR8)

Here are the non functional requirements.

- As a system admin, I want the system to handle an average of 10,000 logged AI usage records at any given time. so that I can perform meaningful statistics. (NFR3)

- As a system admin, I want the system to store the data in a database, so that I can easily retrieve, update and delete information.(NFR5)

1.2 Prompt Strategy

We are building an "AI Guidebook System" web application to support students in everyday academic work, when generative AI tools are embedded in everyday study practices. We aim to provide clear guidance that helps them use AI responsibly. The application will provide the opportunity to declare AI usage in assignments to promote transparency. This will be done through a form in which the student will declare which AI tools he has used and for what purpose. The data collected through the form will be stored in a database.

When prompting for a new feature, attach a detailed list of design choices as well as the functional and non-functional requirements for this set of the system.

The application should be a full-stack web architecture, with React for frontend, and Node.js + Express for backend. For the architecture, the application should follow a monolithic approach. When prompting, technical details and requirement should be clearly specified. The database design should be specified and every attribute should be provided.

As far as prompting patterns are concerned, role-base pattern and chain of thought will be used. The role should be a senior full-stack software engineer with experience in clean code, modular design and security best practices. Frontend experience with HTML, CSS, TypeScript and React, backend experience with Node.js and Express and database experience with PostgreSQL and MySql. When using the chain of thought pattern, the expected reasoning of the model should be stated clearly and detailed and with required observable artifacts (API contract, schema, test plan, validation rules, error handling).

Additional technical requirements can be added by the developers and specified in the prompt. These can include file storage, database design, error handling and input validation decisions.

The result of the prompt should be an initial version of the code, as well as an overview of the file and folder structure and an explanation of the architectural and design choices. This result should be reviewed and iterative refinement should be applied if any issues are found. If there are any constraints on the response format or contents, they should be clearly stated in the prompt.

1.3 Example Prompt

Role description:
You are a senior full-stack software engineer with experience with

- Frontend: HTML, CSS, TypeScript, React

- Backend: Node.js, Express
- Databases: PostgreSQL, MySql
- Software engineering practices: clean code, modular design, security best practices

Here is a description of the project context.

We are building a "AI Guidebook System" web application to support students in everyday academic work, when generative AI tools are embedded in everyday study practices. We aim to provide clear guidance that helps them use AI responsibly. The application will provide the opportunity to declare AI usage in assignments to promote transparency. This will be done through a form in which the student will declare which AI tools he has used and for what purpose. The data collected through the form will be stored in a database.

Act like a professional developer working in a real team. Respond and help me design and implement my web application. When you help me, there are some guidelines that you should follow.

- You should always explain your reasoning briefly before writing code, thinking step by step.
- When you are producing code, you should always produce only one file of code at a time and comment them properly.
- Produced files should have a clearly stated file name at the top (e.g., '/server/index.js').
- If you miss any information, ask for clarification before coding.
- You should not skip any steps in the development process.

Technical requirements:
The web application should use React and TypeScript for the frontend, as well as Node.js and Express for the backend and MySQL for database.

Application description:
We are focusing on the assignment declaration form of the application. The frontend should consist of two pages, one containing the form to be filled out by the user and the other contain a home page that will load at the start of the system and after the form is submitted. The home page will contain all forms submitted, as well as the "create new form" link. When starting the application, the landing page is the homepage with already submitted forms from the database. After the submission of the form, the user is redirected to the homepage where the newly submitted form is present.

User stories:
The requirements for the assignment declaration form feature are listed below.

- As a student, I want to declare AI usage in assignments by filling out a form, so that I can ensure compliance with the ethical guidelines provided by the institution.

- As a student, I want to specify specific tools used in each assignment and their purpose, so that I can achieve transparency in my work.
- As a student, I want to be able to select the tools I used during assignment declaration from a list (ChatGPT, GitHub Copilot, Claude, Grammarly, Turnitin, DALL-E, data analysis tools), so that I can perform the declaration quickly.
- As a student, I want to have the option to declare AI tools that are not present in the list during assignment declaration, so that I can feel free to use the tools that are most useful to me while still being transparent.
- As a student, I want to declare the content generated by AI in my assignments, so that I can be transparent in my work.
- As a student, I want to attach screenshots or examples of AI-generated content to my declaration, so that I can provide clear documentation.
- As a system admin, I want the system to handle an average of 10,000 logged AI usage records at any given time. so that I can perform meaningful statistics.
- As a system admin, I want the system to store the data in a database, so that I can easily retrieve, update and delete information.

Additionally, the code should satisfy the following technical requirements:

- The image files uploaded through the form should be stored in a directory on the server filesystem. The database should store a path to the file together with the rest of the usage record.
- The system should not allow the user to upload files that are not images or that are larger than 5MB in size.
- The text field for the "Other" option in the AI tool selection in the form should allow only strings of letters (lowercase and uppercase), numbers and punctuaction of maximum length of 50 characters.
- The database should include one table with the following schema:

```
CREATE TABLE IF NOT EXISTS ai_declarations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_name VARCHAR(255) NOT NULL,
  assignment_title VARCHAR(255) NOT NULL,
  ai_tool VARCHAR(255) NOT NULL,
  usage_purpose TEXT NOT NULL,
  ai_content TEXT NOT NULL,
  screenshot_path VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_name (user_name),
  INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Define a structured development strategy for the application and proceed with the code development, ensuring all requirements and guidelines are followed. The result should be a working full-stack web application.

Chapter 2

Task 2.2. AI-Based Code Generation

The code created for Task 2.2 is found on the public repository https://github.com/Raffaella-Corcione/TDT4242_group03.

2.1 Installation and configuration

This chapter includes the installation and configuration of the required libraries and dependencies. Before beginning the installation, ensure you have Node.js, MySQL and Git installed on your system.

2.1.1 Environment Configuration

In the project root folder `/files`, create a file named `.env`. Insert the following configuration:

```
# Database Configuration
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=[your_mysql_password_here]
DB_NAME=ai_guidebook_db
DB_PORT=3306

# Server Configuration
PORT=5000

# Client URL (for CORS)
CLIENT_URL=http://localhost:5173
```

Update the `DB_PASSWORD` field to match your local MySQL password.

2.1.2 Installation Steps

Navigate to the root directory `/files`.

```
cd files/
npm install
```

After install complete, navigate to `/client` folder.

```
cd client/
npm install
```

2.1.3 Running the Project

To run the application, navigate to the `/files` folder.

```
cd files/
npm run dev
```

2.2 Revision of AI-generated code

The code was generated by GitHub Copilot, with the prompting strategy described in Task 2.1. After retrieving the generated code from the AI, it was cloned into our GitHub repository. The application worked as expected without any major errors. The application started without any problems, and the homepage and declaration form worked as expected. However, there were some minor adjustments done manually.

One adjustment that was made, was fixing the counter for the number of declarations submitted. There was a small bug, which caused the counter to display the wrong value. The generated code form GitHub Copilot was as following.

```
<h2>Submitted Declarations ({declarations.length})</h2>
```

This line of code was changed manually to:

```
<h2>Submitted Declarations ({Object.keys(groupedDeclarations).length})</h2>
```

2.3 Explanation of the codebase structure and design decisions

The application architecture follows a three-tier approach. The three layers are the presentation layer (Frontend - React/TypeScript), the application layer (Backend - Node.js/Express) and the data layer (MySQL Database).

2.3.1 Frontend components

The presentation layer is confined in the directory `client/src/`. Its components are:

- `main.tsx`: this is the entry point of the application. Its purpose is to bootstrap the React application, render the root App component into the DOM and call App.tsx.
- `App.tsx`: this is the routing controller. It manages navigation between pages, sets up the React Router and defines two routes: `/` (home) and `/new-declaration` (form). It then renders either `HomePage` or `DeclarationFormPage` based on URL.
- `types/index.ts`: Type definition file. Contains centralized TypeScript interfaces that define database record structure, form input structure and predefined AI tool list for the form. Used by all components for type safety.
- `HomePage.tsx`: displays all submitted declarations fetched from the API (`GET api/declarations`) Declarations are grouped by assignment and displayed in cards. There is also a link to create new declarations. The state management includes an array of all records, a loading state and the error messages. This component calls backend API on mount (`useEffect`) and navigates to `DeclarationFormPage` via React Router link.
- `DeclarationFormPage.tsx`: Form component, collects AI usage declaration data. It handles client-side parameter validation, file upload, data submission and it navigates back to homepage on success.

2.3.2 Backend components

The backend of the application is contained in the `server/` directory.

- `index.js`: Server Entry Point. Initializes and configures the Express server, loads environment variables, configures the middleware, registers API routes, initializes database, starts the HTTP server on port 5000. It also catches and logs all server errors.
- `config/datab\input{sections/30-code-review }ase.js`: this is the database manager of the application. It manages MySQL connection and schema.
- `routes/declarations.js`: API endpoints to handle declaration CRUD operations. The endpoints are:
  - `GET /api/declarations`: fetches all declarations from the database. Returns a JSON.
  - `POST /api/declarations`: accepts form data with file upload and handles server-side parameter validation. It saves uploaded images to `/uploads` directory, creates the new database records and returns the newly created records.

2.3.3 Example data flow: form submission process

Here is an explanation of the complete process of the assignment declaration, with the involved components and their interactions.

1. User fills out form (`DeclarationFor,mPage.tsx`): the user interacts with the form and inserts all the required fields. Each input updates `formData` state with `handleInputChange` or `handleToolChange`.

2. User clicks "Submit Declaration" button: the `handleSubmit(e)` function is called. The default browser behavior is blocked and the function performs validation checks on the data: all the required fields must be not empty, at least one AI tool must be selected or a custom tool must be entered, and the image must be in one of the allowed formats and with an appropriate size.

3. Prepare data for submission (`DeclarationFormPage.tsx`): a `formData` object is created and its fields are filled with the data entered in the form.

4. Send HTTP POST request (`DeclarationFormPage.tsx`): a network request is sent to the server - `POST http://localhost:5000/api/declarations`

5. Backend receives request (`server/index.js` -¿ `routes/declarations.js`): the request flows through the middleware, which validates its origin, calls the router and processes file upload.

6. Validate request data (`routes/declarations.js`): server-side parameter validation is performed before the data is saved.

7. Save file path: if a picture was uploaded, it is saved in the `/uploads/` directory, and its path is saved to be entered into the database record.

8. Insert into database (`routes/declarations.js`): one record per AI tool is created with an INSERT query. Multiple insertions are executed asynchronously and in parallel with the use of Promises.

9. Fetch newly created records: a SELECT query for the newly inserted records is executed, so they are retrieved from the database.

10. Send response to Frontend: an HTTP response is sent with status 201. A JSON object containing a message and the newly created data is delivered to the client.

11. Frontend handles response (`DeclarationFormPage.tsx`): if the response is successful, the application navigates to the home page to display the new list of declarations (via React Router navigation). Otherwise, an error is displayed.

12. HomePage loads and displays data: the `useEffect` function fetches the complete list of declarations (`GET /api/declarations`), triggering a database query on backend.

13. Grouping and rendering: the home page loads as normal.

2.4 Screenshots of successful execution

Here are some screenshots demonstrating successful execution of the initial version of the application.

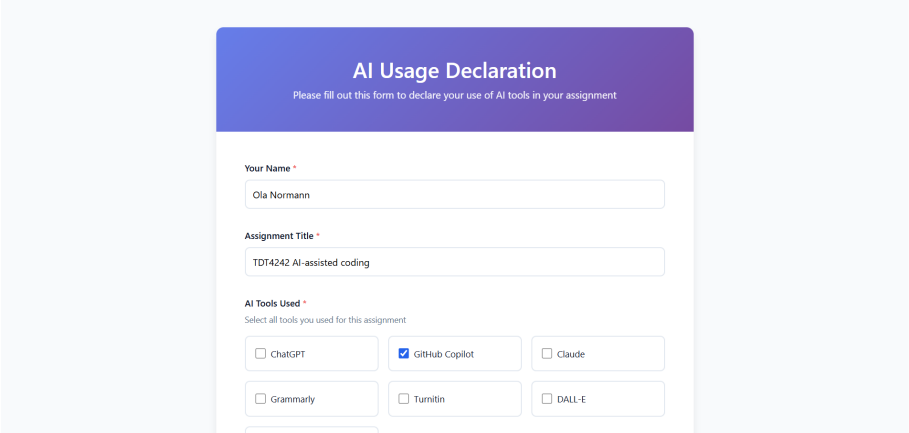


Figure 2.1: The declaration form, first part.

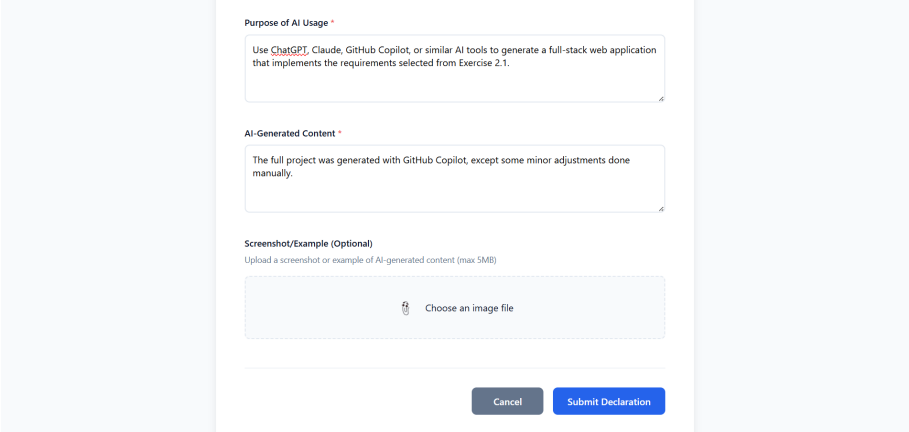


Figure 2.2: The declaration form, second part.

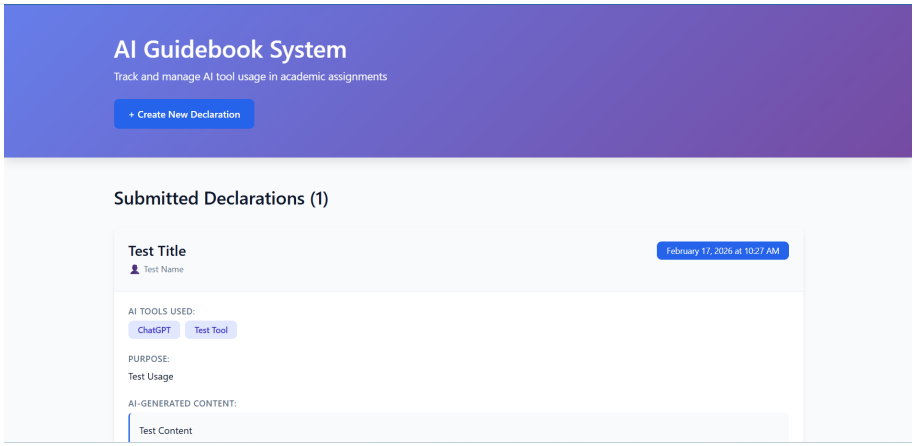


Figure 2.3: The initial homepage.

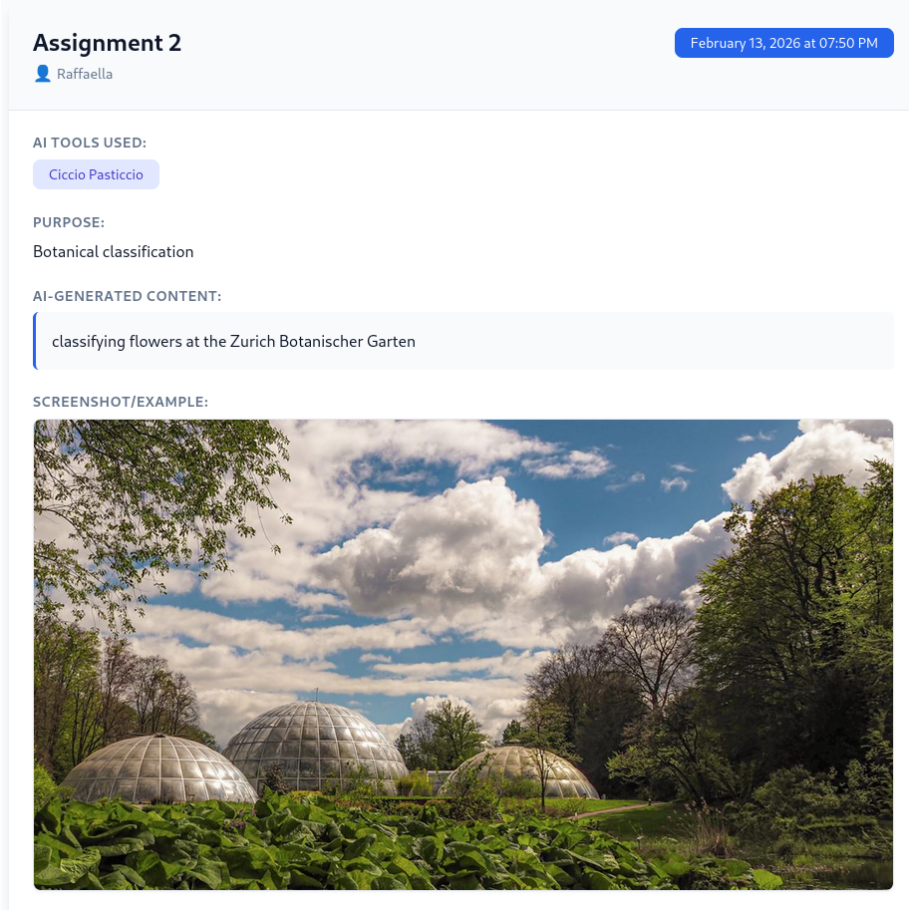


Figure 2.4: A submitted declaration.

Chapter 3

Task 2.3. Code Review

In order to identify code smells and execute the code review, we have decided to focus on manual and AI-assisted analysis. To identify code smells manually, the application architecture and data flow were inspected. The code of the involved components was analyzed to find structural issues and potential problems. Whenever AI-assisted review was employed, the prompt that was used is included in this report.

3.1 Large Class

The component `DeclarationFormPage` is an example of the Large Class code smell. At 392 lines of code, it attempts to handle too many responsibilities simultaneously, including data validation, file handling, form construction, API submission, navigation, and UI rendering.

With its many variables and responsibilities, the component becomes harder to read and maintain. Large classes often lead to chaos, reduced readability, and duplicated logic. This can be improved by applying refactoring like *Extract Component* or *Extract Subclass*, which split `DeclarationFormPage` into smaller, more focused, and cohesive units [1].

This smell was identified through manual code review, as shown in Figure 3.1.

3.2 Duplicated Code

The `DeclarationFormPage` component has some duplicated code, in the form input sections. The blocks `<div className="form-group">` are repeated with small differences in the parameters.

By repeating code, these blocks make the code longer and harder to read. A possible way to solve this, is to create a reusable `FormField` component that takes props like `id`, `name` and `value`. Then, each input can be rendered with just one line of code, making the form shorter and easier to maintain [1].

This smell was identified through manual code review, as shown in Figure 3.2.

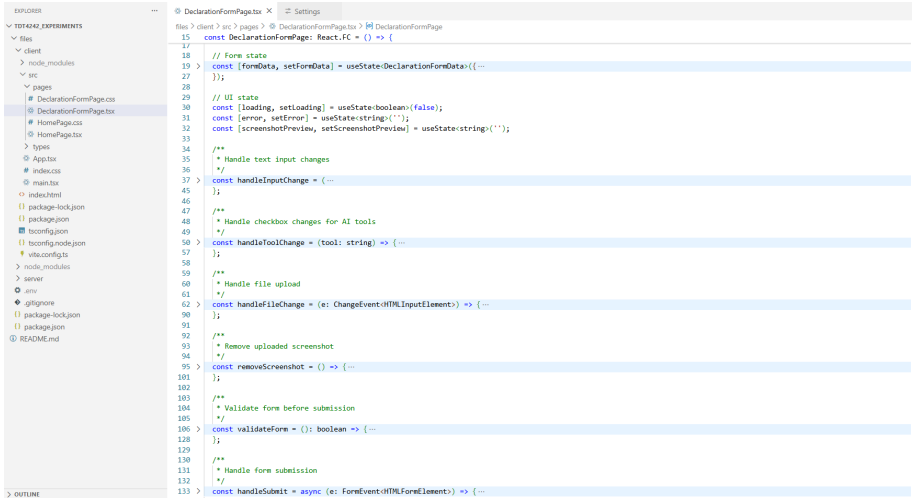


Figure 3.1: The `DeclarationFormPage` showing the Large Class code smell

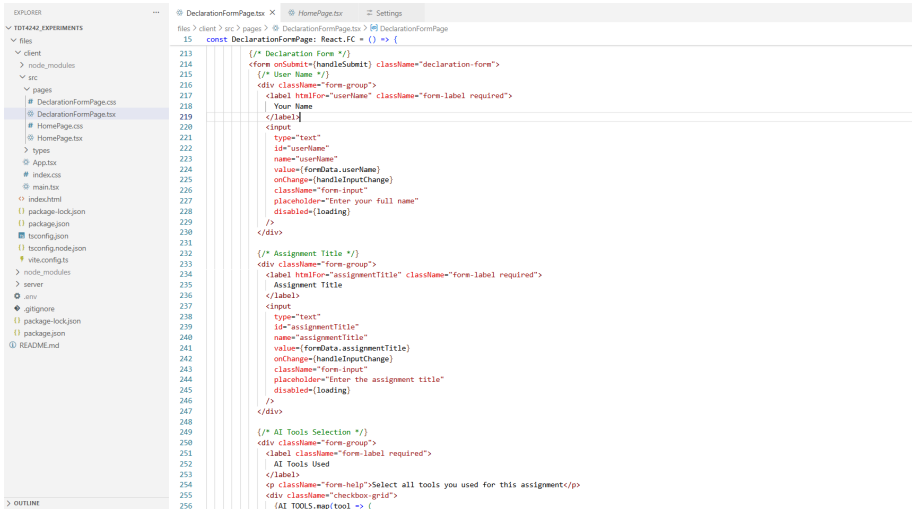


Figure 3.2: The `DeclarationFormPage` showing the Duplicated Code code smell

3.3 Primitive Obsession

All the variables and the parameters used in the code are expressed as primitive types, most frequently strings. Parameter validation at form submission, both server- and client-side, happens only by checking for string non-emptiness. Strings have no semantic meaning, and do not encode domain knowledge. This is an example of "Primitive Obsession" code smell. However, due to the small scope of the application, this solution is acceptable. In fact, parameters such as username, assignment title, custom AI tool, generated AI content and AI usage purpose have no predefined length, structure or format constraints (apart from being not null) in the requirements of the system. Therefore, the type system is left as is. This smell was found through manual code review, by inspecting the input validation process and data flow.
The main code sections affected by the smell are the form validation functions in the client and server code.

The client-side parameter validation is done as follows:

```
  const validateForm = (): boolean => {
  if (!formData.userName.trim()) {
    setError('Please enter your name');
    return false;
  }
  if (!formData.assignmentTitle.trim()) {
    setError('Please enter the assignment title');
    return false;
  }
  if (formData.aiTools.length === 0 && !formData.customTool.trim()) {
    setError('Please select at least one AI tool or enter a custom tool');
    return false;
  }
  if (!formData.usagePurpose.trim()) {
    setError('Please describe the purpose of AI usage');
    return false;
  }
  if (!formData.aiContent.trim()) {
    setError('Please describe the AI-generated content');
    return false;
  }
  return true;
};
```

In the server, we find very similar checks:

```
  if (!userName || !assignmentTitle || !aiTools || !usagePurpose || !aiContent) {
    // If file was uploaded but validation failed, delete it
    if (req.file) {
      fs.unlinkSync(req.file.path);
    }
    return res.status(400).json({
      success: false,
      message: 'All fields except screenshot are required'
    });
  }
```

Security note: even without strong type checking, the code is preventing SQL injection attacks through the use of prepared statements.

3.4 Dead/unreachable code

This smell was identified with the AI-assisted code analysis tool GitHub Copilot. The identification was based on the following initial prompt:

> "Help me identify dead or unreachable code in the codebase. Follow this reasoning process: examine both the client and server code, and find any functions

> or components that are unreachable from the rest of the code and so can be removed. Report them one by one and explain why they are unreachable and what their intended purpose was. Recommend an approach to fix the problem. If there are no dead or unreachable code snippets, report that the code does not exhibit this smell."

The result of the analysis indicated that this smell is not a major issue in the codebase. However, the AI-assisted review highlighted a couple of minor issues: first, there is an unused type `Export AITool` in the files/client/src/types/index.ts TypeScript file. This type is exported but it's never imported or used anywhere in the codebase. Its intended purpose was to provide type-safe references to AI tool names, restricting variables to only valid predefined tool names from the AI_TOOLS array. It is declared as follows:

```
export type AITool = typeof AI_TOOLS[number];
```

The recommended fix to this issue is to use this type in the DeclarationFormData structure and in the DeclarationFormPage code to improve code readability (over the use of a string array).

Secondly, there is an nused CSS Utility Class `.text-center`, located at files/client/src/index.css and declared as follows:

```
.text-center {
    text-align: center;
}
```

This class is never used, as all components use inline styles or specific classes. It was added for quickly centering text. The recommended approach is to remove it.

3.5 Duplicated logic

This smell was analyzed with the help of GitHub Copilot, basing on the following prompt:

> "Help me identify duplicated logic in the codebase. Follow this reasoning process: examine both the client and server code, and find any functions or components that implement the same logic and can be simplified. Report them one by one and explain what code is duplicated and the reasoning behind it. Recommend an approach to unify and centralize the duplicated logic. If there are no duplicated code snippets, report that the code does not exhibit this smell."

The most obvious instance of duplicated logic that was highlighted by the analysis is related to user input validation: the client and the server both validate the same business rules (userName must not be empty, assignmentTitle must not be empty, at least one AI tool must be selected, usagePurpose must not be empty, aiContent must not be empty). The involved code client-side is the following:

```
  const validateForm = (): boolean => {
  if (!formData.userName.trim()) {
    setError('Please enter your name');
    return false;
  }
  if (!formData.assignmentTitle.trim()) {
    setError('Please enter the assignment title');
    return false;
  }
  if (formData.aiTools.length === 0 && !formData.customTool.trim()) {
    setError('Please select at least one AI tool or enter a custom tool');
    return false;
  }
  if (!formData.usagePurpose.trim()) {
    setError('Please describe the purpose of AI usage');
    return false;
  }
  if (!formData.aiContent.trim()) {
    setError('Please describe the AI-generated content');
    return false;
  }
  return true;
};
```

whereas server-side we have the following code:

```
if (!userName || !assignmentTitle || !aiTools || !usagePurpose || !aiContent) {
  // If file was uploaded but validation failed, delete it
  if (req.file) {
    fs.unlinkSync(req.file.path);
  }
  return res.status(400).json({
    success: false,
    message: 'All fields except screenshot are required'
  });
}
```

The reasoning for this duplication is both user experience (the user gets immediate feedback about missing parameters before the data is sent to the server) and security (no data should be inserted into the server without being checked server-side). However, this solution causes low mantainability (in case of change of validation logic) and inconsistency risk (the client may allow something the server rejects). The recommended solution is a shared validation schema that both client and server can use. We do not think this is necessary, as the validation logic is very simple and it's present only for the form submission. The same goes for the file validation logic.

Additionally, the analysis emphasized a potential issue in the error handling logic: the error response structure is the same across the entire server code, and it is manually

constructed each time. This could make testing harder, as it is more difficult to make sure that all errors follow the same format. The recommended solution is to centralize the error response helpers, adding a new file at files/server/utils/responses.js and changing the declarations.js file accordingly. As an example, we have reported below the before and after error handling regarding the declaration fetch at home page loading:

```
catch(error){
    console.error('Error fetching declarations:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to fetch declarations',
        error: error.message
    });
}
```

```
catch(error){
    console.error('Error fetching declarations:', error);
    return serverError(res, 'Failed to fetch declarations', error);
}
```

The `serverError` function is declared as follows:

```
function serverError(res, message = 'Internal server error', error = null) {
    return errorResponse(res, message, 500, error);
}
```

A similar approach is used for all the other types of error.

3.6 Other

By looking at the data flow for the assignment submission form, we noticed a flaw in the managing of data sent by the server as a response to the client after form submission. Initially, when a client submitted a declaration, the server responded with an object that contained the status of the submission (successful/not successful), a message, and a list containing all the new records that were created, making a SQL query to the database (file `server/routes/declarations.js`):

```
// Fetch the newly created records
const [newRecords] = await pool.query(
  `SELECT * FROM ai_declarations
   WHERE user_name = ? AND assignment_title = ?
   ORDER BY created_at DESC
   LIMIT ?`,
  [userName, assignmentTitle, toolsArray.length]
);
```

```
res.status(201).json({
```

```
  success: true,
  message: `Successfully created ${toolsArray.length} declaration(s)`,
  data: newRecords
});
```

however, when the client reloaded the home page after the successful form submission, the new records were fetched again, making the first query useless. So it was removed, so that the new object that is returned to the client is the following:

```
res.status(201).json({
   success: true,
   message: `Successfully created ${toolsArray.length} declaration(s)`,
});
```

Then the home page loading is performed as normal. We preferred this solution over an optimistic UI update of the HomePage with only the new records for two reasons: it was a simpler modification that did not require implementation of complex state management, and it allowed to also fetch any new declarations from other users in a single update.

Bibliography

[1] Martin Fowler. *Refactoring - Improving the Design of Existing Code.*